ELI — Inference And Hardware

ELI v2.3.13

August 2026

Contents

ELI Inference & Hardware Boot

How ELI loads a model, talks to it, and adapts to whatever machine it's on. The inference path is model-agnostic (see memory `eli-model-agnostic`); the boot path is hardware-adaptive. Files in `eli/cognition/` and `eli/core/`.

Inference (`cognition/gguf_inference.py`, 2.7k LOC + `inference_broker.py`)

- **Model resolution (`get_model_path`)**: `ELI_GGUF_MODEL_PATH` env → `model_path`/`custom_model_path`/`bundled_model_p` settings keys. No baked model; empty default.
- **`load_model(force_reload)`**: resolves `n_ctx` (env → settings → `config.get_gguf_n_ctx()`), caps it to `vision_coresident_text_ctx` when a co-resident vision model is loaded; resolves `n_gpu_layers` similarly.
- **Graceful GPU-layer fallback**: if context allocation fails at the requested layer count, it retries with fewer layers / without flash-attention rather than crashing — keeps the model + n_ctx and degrades GPU offload instead.
- **Chat templating is family-aware**: `_is_mistral_model` / `_is_chatml_model` / `_is_llama_model` sniff the filename to pick the right prompt format. This is *adaptation* to whatever model you load (like the ctx table), not a hardcoded model — but it is filename-based, so an unrecognised naming scheme falls back to a default template.
- **Serialization**: all calls hold `_LLM_CALL_LOCK` (a native RLock from `runtime/native_locks`) — llama_cpp is not safe under concurrent calls; this is also what vision hot-swap and the ambient daemon coordinate on.
- **Live control**: `get_live_runtime_override`, `unload_model`, `reload_model` let the GUI swap models / change settings without a full restart.
- **`InferenceBroker`** (`inference_broker.py`): the higher-level `infer()` / `gguf_ready` abstraction the orchestrator, engine, and ReAct loop call, so callers don't touch `gguf_inference` directly.

Hardware profiling (`core/hardware_profile.py`, 1232 LOC)

Free-VRAM-aware sizing: - `HardwareProfile` dataclass tracks **free** vs total VRAM (free is what matters for whether a profile actually loads). - `_kv_cache_mb(n_ctx, n_layers)` — KV-cache cost. - `_compute_graph_reserve_mb(n_ctx, batch)` — the **model-agnostic** compute buffer estimate (`256MB + 24MB/1K ctx + 1.5MB/batch`), reserved so a profile that loads cleanly doesn't then hard-crash on the first decode when the lazy compute buffer pushes VRAM over the limit. (This was a real crash class.) - `_layers_for_size`, `ModelRecommendation` — pick offload layers from model size and free VRAM.

Boot optimizer (`core/startup_hardware_optimizer.py`, 655 LOC)

Runs at startup, writes `artifacts/runtime_hardware_profile.json`: - `detect_ram_gb`, `detect_cpu_name`, `detect_nvidia_gpus` (+ `detect_other_gpus` fallback), `select_gpu`. - `find_model(settings)` — locate the GGUF. - **`train_ctx_for_model(model_path)`** — the filename→context table (deepseek/llama-3.1/phi/gemma-2 → 128K; qwen2.5/mistral-7b → 32K; older → 8K; unknown → 32768). This is the core of model-agnostic context sizing. - `estimate_layers`, `layer_mb` — VRAM-fit layer count.

Settings (`core/runtime_settings.py`, 1134 LOC)

- `DEFAULTS` (the full settings schema) + `ENV_TO_KEY` (env-var overrides).
- `load_settings` / `save_settings` / `update_settings`.
- **Redistribution-aware**: `_migrate_legacy_keys` (schema evolution), `_resolve_relative_model_paths` + `_heal_model_paths` (fix stale absolute paths when the project moves machines), and **`_portable_settings_for_sto`** (strip machine-specific values before storage). The intent to keep settings portable across machines is built in — though personal values like `user_name` can still end up tracked (see the settings.json commit note).

Paths (`core/paths.py`, 601 LOC)

Dev-vs-packaged path resolution: `is_frozen` / `_is_dev_mode`, `_find_project_root`, then `data_dir/config_dir/cache_dir/m` `db_dir/artifacts_dir/user_db_path/agent_db_path/memory_db_path`. Source checkouts use project-local `artifacts/+config/`; packaged installs use platformdirs. One import surface (`get_paths`) so nothing hardcodes locations.

Honest assessment

- **Strong:** genuinely adaptive and agnostic — free-VRAM-aware sizing, the compute-buffer reservation that prevents first-decode crashes, graceful GPU-layer fallback, filename→ctx adaptation, env/settings/relative-path healing for moving between machines, and a single broker + lock so concurrency is correct. This is mature, hard-won infrastructure.
- **Weak / watch:**
  1. **Filename-based family detection** (chat template + ctx) is fragile: a model with an unconventional filename gets a default template + 32768 ctx, which can be wrong (mis-templated output, or ctx overflow on a small model). A metadata/GGUF-header probe would be more robust than string matching.
  2. **Settings sprawl** — `DEFAULTS` is large with several overlapping keys (`n_gpu_layers`/`gpu_layers`, `n_ctx`/`context_size`) and migration logic, echoing the schema churn seen in `memory/`.
  3. VRAM heuristics are empirically tuned around an 8GB card; very different hardware (24GB+, or CPU-only) leans on the conservative fallbacks rather than tuned values.
  4. `_portable_settings_for_storage` exists but isn't fully preventing personal values (e.g. `user_name`) from being persisted/committed — worth tightening for redistribution.

Update — 2026-06-09 (STT no longer starves the main model)

- faster-whisper preloaded on CUDA **before** the main GGUF autotuned, claiming ~2 GB so on an 8 GB card the main model only got `gpu_layers=11` (free_vram=4083 MB) → 23–59 s turns. STT is now VRAM-aware (`local_whisper_stt`: GPU only on ≥12 GB cards, else CPU), so the main model reclaims the GPU — `gpu_layers=99 / ctx=20480` with free_vram ~7 GB, turns back to 1–12 s. `ELI_WHISPER_DEVICE` overrides; `ELI_WHISPER_GPU_MIN_MB` tunes the threshold. See `perception.md`.

Update — 2.3.7 (vendor parity, stated honestly)

Where parity already held

`hardware_profile.py` is genuinely cross-vendor and always was: NVIDIA via `nvidia-smi`, then a kernel-driver fallback, then **AMD via `rocm-smi`**, then **AMD via the stock `amdgpu` sysfs** (`/sys/class/drm/card*/device/mem_info`, the common desktop case where ROCm is absent), then **discrete Intel Arc**. All of them populate the *same* `HardwareProfile` fields, so smart-fit GPU-layer allocation is identical whatever the card. `install.sh` builds llama-cpp with ROCm/hipBLAS, falling back to Vulkan, falling back to CPU. KV cache and GPU offload are at parity.

Where it leaked

The leak is not the profiler — it is the **~15 sites that bypass it** and re-implement `nvidia-smi` directly. Consequences on an AMD or Intel box:

- `perception/local_whisper_stt.py` — `_gpu_total_mb()` shells to `nvidia-smi`, gets 0, and pins Whisper to CPU regardless of the card. (CTranslate2 has no ROCm backend, so the *outcome* is currently correct, but the reasoning is not, and the user is told nothing.)
- `runtime/self_status.py`, `runtime/truth_report.py`, the executor's GPU report — all report "GPU telemetry unavailable", i.e. **ELI disowning hardware it has**. That is the same failure family as the false-self-denial guards elsewhere in the codebase: fabricating a capability and denying a real one are both dishonesty.

The fix direction is to route those sites through `hardware_profile`, which already knows the answer, rather than adding more vendor detection.

Piper

Worth knowing because it is often assumed otherwise: Piper is a **subprocess binary** (`tts_piper/piper`), not onnxruntime-in-Python, and the shipped build is CPU-only — for every vendor. There is no NVIDIA advantage to close there; parity already holds, at CPU.

LoRA training device selection

`eli/learning/lora_trainer._accelerator()` is the model for how the rest should read. torch routes ROCm through the `torch.cuda` API — a HIP build answers `torch.cuda.is_available()` — so one code path serves NVIDIA and AMD, and the vendor is read from `torch.version.hip` and reported as what it is rather than as "CUDA on a Radeon". Apple (mps) and Intel (xpu) are detected and honestly marked as unable to run that trainer. See `learning.md`.